

Tecnicatura:

**TECNICATURA SUPERIOR EN CIENCIA DE
DATOS E INTELIGENCIA ARTIFICIAL**



|Segundo Cuatrimestre - 2do Año

|MÓDULO - Práctica PROFESIONALIZANTE I

|Docente - ACCIETTO Dianiel

|Grupo - N²-EFMG Solutions

<https://github.com/Nieves862/NN-EFMG-Solutions>

Ciente: Escuela PRoA (Río Tercero)

Desarrollador: N²-EFMG

Fecha: Septiembre de 2026

Integrantes & Roles:

- Valeria Nieves Villalba (Business Analyst)
- Florencia Santarena (Data Scientist / Backend)
- Mariela Suarez (Machine Learning Engineer)
- Juan Gabriel Cabrera (Full Stack QA Engineer)
- Nicolás Allende Olmedo (Project Manager)
- Emanuel Guaraz (IA Engineer)

01. Exploración y Justificación del Dato

◆ ¿De dónde sale el dato?

El dato se origina en el entorno físico de la Escuela PProA Río Tercero mediante una arquitectura IoT local basada en Arduino UNO R3. Previo al armado físico, el circuito y la lógica de control fueron simulados y validados en la plataforma Tinkercad. La captura ambiental física se realiza a través de dos sensores principales:

- **Sensor DHT11:** Capta en tiempo real las variables de temperatura (en °C, con precisión entera) y humedad relativa (%).
- **Sensor MQ-2 (Gas / Humo):** Módulo electroquímico con cápsula metálica reticulada de color plateado y anillo identificador naranja, utilizado para detectar la presencia de gases combustibles (GLP, propano, metano), humo y vapores en el ambiente. Mide la concentración traduciendo la variación de resistencia del sensor a un valor analógico (`analogRead(A0)`) procesado por el Arduino UNO y convertido a Partes Por Millón (PPM) estimadas en la base de datos `estacionmeteorologica_proa`.

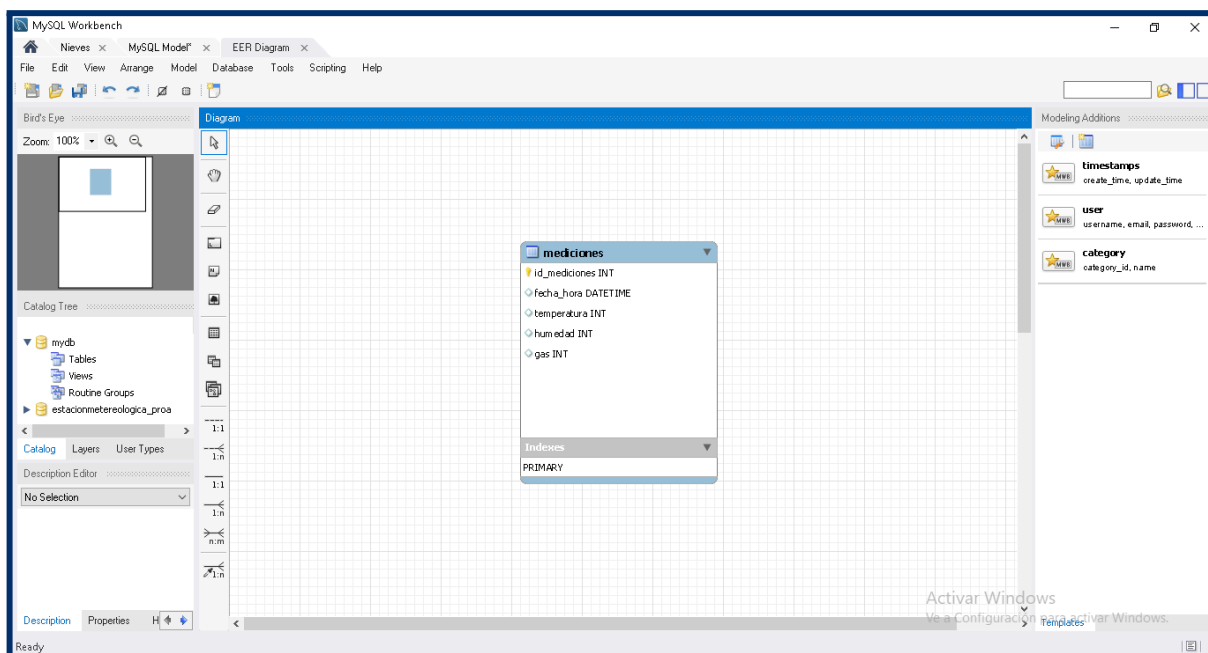
El microcontrolador (programado en C++ mediante Arduino IDE) empaqueta las lecturas en una trama de texto delimitada por comas (`Humedad, Temperatura, Gas\r\n`) y la transmite cada 2 segundos a 9600 baudios vía USB (puerto COM) hacia la computadora central de ingesta.

`\r` (Carriage Return / Retorno de Carro): Le indica al receptor que mueva el cursor al inicio de la línea.

`\n` (Line Feed / Salto de Línea): Le indica al receptor que avance una línea hacia abajo.

◆ ¿Qué base de datos, planilla o fuente usaron?

Para la captura y persistencia de la información, el sistema toma como fuente de origen una transmisión en vivo mediante el puerto de serie físico USB (COM), gestionado en Python 3.12 con la librería `pyserial` dentro de Visual Studio Code. Desde allí, el flujo de datos se canaliza hacia una base de datos relacional MySQL Server administrada a través de MySQL Workbench, donde las lecturas se insertan de forma estructurada en la tabla `mediciones` del esquema `estacionmeteorologica_proa`.



◆ ¿Cómo está organizado y qué decidieron limpiar, descartar o transformar?

A. El dato fue estructurado relacionalmente en MySQL Workbench para optimizar espacio. La tabla `mediciones` contiene los siguientes campos:

- `id_mediciones`: Llave primaria autoincremental (`INT AUTO_INCREMENT PRIMARY KEY`).
- `fecha_hora`: Marca temporal automática generada por el motor relacional (`DATETIME DEFAULT CURRENT_TIMESTAMP`).
- `temperatura`: Guardado como entero (`INT`) debido a que el sensor DHT11 reporta variaciones de 1 en 1 grado.
- `humedad`: Guardado como entero (`INT`) para ocupar el mínimo espacio posible en disco.
- `gas`: Incorporado mediante la sentencia `ALTER TABLE mediciones ADD COLUMN gas INT AFTER humedad` para almacenar la concentración analógica del sensor MQ-2 en valores enteros (PPM).

B. Para cumplir con los criterios de la **HU-03 (Verificación de Comunicación Serial)**, en el entorno de Visual Studio Code (Python) se programó una canalización defensiva:

1. **Decodificación de Bytes Crudos**: El puerto COM transmite bytes (`b'57,24,120\r\n'`). Se aplica `.decode('utf-8', errors='ignore').strip()` para remover saltos de línea, retornos de carro y prevenir que el script falle ante caracteres basura al conectar el USB.
2. **Filtro de Descarte de Tramas Corruptas**: Si el sensor DHT11 falla o se desconecta un cable Dupont, el Arduino transmite `"ERROR,ERROR,ERROR"`. El script Python evalúa `if "ERROR" not in linea`: y valida que la trama tenga exactamente 3 elementos (`len(datos) == 3`). Si la lectura llega incompleta o nula por ruido eléctrico, es descartada en memoria volátil, evitando inyectar errores dentro de MySQL.
3. **Parseo y Asignación Categórica de Gas**: Dado que el sensor MQ-2 es un dispositivo analógico de espectro amplio que no distingue la firma química exacta del gas, la clasificación del riesgo se resuelve en la capa de software (Python/SQL) modelando la severidad según rangos de PPM:
 - `< 250 PPM`: *Aire Normal / Estado Óptimo*.
 - `250 - 450 PPM`: *Alerta Media (Humo o Vapores Suaves)*.
 - `> 450 PPM`: *Alerta Crítica (Fuga de Gas / Alta Concentración)*.
4. **Transformación Visual e Indicadores en Consola**: Se mapean los tipos a `int` y se renderizan marcos interactivos utilizando la librería `colorama` (mostrando en verde condiciones óptimas, en amarillo alertas medias y en rojo alertas críticas de gas o temperatura).
5. **Control de Flujo de Red**: Se asegura la reconexión e impresión continua del estado `[CONECTADO]` / `[DESCONECTADO]`. Mediante la propiedad `autocommit=True` en el conector de MySQL, se garantiza la inserción inmediata del registro sin bloqueos de transacciones.

◆ ¿Por qué eligieron trabajarlo de esa manera?

Elegimos esto porque:

- **Calidad de Datos en la Cadena de Ingesta**: Preferimos filtrar las tramas defectuosas en el buffer de Python (antes de ejecutar la sentencia `INSERT`) evita contaminar las series temporales de la base de datos de la escuela.
- **Separación de Responsabilidades (Hardware vs. Software)**: Al Arduino lo dejamos que se encargue únicamente de la lectura/actuación física (pantalla LCD, LEDs y Buzzer) y delegar a Python/MySQL la clasificación por software de los tipos de aire y alertas, optimiza el procesamiento del microcontrolador y da flexibilidad para ajustar umbrales sin reflashar el Arduino.
- **Eficiencia de Almacenamiento**: Guardar variables numéricas completas como `INT` optimiza el tamaño de la base de datos en MySQL Workbench, permitiendo soportar lecturas continuas sin sobrecargar los recursos informáticos del colegio.

- **Validación Visual de QA:** La representación visual con colores en la consola interactiva de Python permite a docentes y alumnos validar el acoplamiento del hardware previo a la etapa de consumo vía WhatsApp.

02. HU-AGRO-01: Persistencia Automática y Visualización Online de Datos Climáticos Rurales

Con el fin de dar continuidad al proyecto atendiendo las necesidades de los trabajadores del sector Rural del éjido municipal de Río Tercero y alrededores, elaboramos la historia de usuario correspondiente, siguiendo los lineamientos respecto a los criterios de aceptación y el método I.V.E.S.T aplicables a los pain point del Usuario propiamente dicho.

◆ 1. Historia de Usuario

- **Como** productor agrícola de la zona de Río Tercero,
- **quiero** acceder a una plataforma en línea que registre automáticamente los datos en tiempo real de la estación meteorológica y me permita consultar gráficos históricos por rangos de tiempo,
- **para** disponer de información precisa del microclima local y tomar decisiones fundamentadas de siembra, riego y prevención, evitando pérdidas económicas derivadas de depender de estaciones distantes.

◆ 2. Criterios de Aceptación

- **Escenario 1: Almacenamiento e Ingesta Automática en Tiempo Real**
 - **Dado** que la estación meteorológica está activa registrando datos en Río Tercero,
 - **Cuando** el sistema recibe una nueva lectura del microcontrolador,
 - **Entonces** el registro se inserta automáticamente en la base de datos MySQL con su marca de tiempo (timestamp) precisa y sin intervención manual.
- **Escenario 2: Visualización de Tendencias Climáticas en la Página Web**
 - **Dado** que existen registros meteorológicos acumulados en la base de datos,
 - **Cuando** el productor agrícola ingresa al sitio web y selecciona un intervalo de tiempo (ej. últimas 24 horas, última semana o rango de fechas personalizado),
 - **Entonces** la página renderiza un gráfico interactivo (de líneas o barras) con ejes etiquetados (Tiempo vs. Temperatura/Variables climáticas) para identificar picos y patrones térmicos locales.
- **Escenario 3: Resiliencia del Almacenamiento en Zona Rural**
 - **Dado** que la conexión a la base de datos central o a internet se interrumpe en el sector rural,
 - **Cuando** el sensor continúa capturando lecturas,
 - **Entonces** el sistema resguarda los datos localmente en un archivo de buffer y los sincroniza automáticamente con la plataforma web al restablecerse la red.

-  Videos y Photos de PRUEBAS

◆ 3. Evaluación bajo el Criterio INVEST

- **I – Independiente:** Es autónoma dentro de la arquitectura web al acoplar el proceso de captura en segundo plano con la vista de consulta en línea.
- **N – Negociable:** Permite negociar los tipos de gráficos, los parámetros visualizados (temperatura, humedad o lluvias) y la frecuencia de actualización del portal. Siempre que se dé prioridad a las visualizaciones de los datos más relevantes para el productor.
- **V – Valiosa:** Resuelve directamente el *pain point* agrícola de Río Tercero al eliminar la incertidumbre sobre microclimas locales y optimizar la toma de decisiones productivas.
- **E – Estimable:** Combina una tarea de persistencia directa en MySQL con el renderizado web mediante librerías de visualización conocidas. Lo que llevaría un desarrollo aproximado de entre 25 a 35 líneas de código.
- **S – Small (Adecuada):** Delimita el almacenamiento automático y la consulta visual dentro de una misma interfaz de usuario. El desarrollo puede realizarse en 2 semanas aproximadamente.
- **T – Testeable:** Se valida mediante pruebas de inserción continua en la base de datos y la verificación del gráfico resultante en el navegador.

03. Informe Breve: Actualización de la Justificación de la Evidencia 1

En la Evidencia 1 se planteó como problema central, la falta de una infraestructura centralizada y automatizada para almacenar e historiar las lecturas de la Estación Meteorológica y Ambiental de la Escuela PROA Sede Río Tercero, lo que provocaba la pérdida de datos al desconectar el hardware y la imposibilidad de realizar análisis estadísticos. La propuesta original justificó la creación de una canalización de ingesta en Python para persistir las variables en MySQL (respaldada por un buffer en CSV) y el desarrollo de un Chatbot de WhatsApp como interfaz de baja fricción para democratizar las consultas de la comunidad educativa y municipal.

En esta segunda etapa, la justificación se fortalece y expande, se confirmó que la captura física vía USB y el procesamiento desacoplado garantizan la integridad de los datos, y se amplió el impacto de la solución para responder también a las necesidades operativas de los productores agrícolas del ejido rural de Río Tercero, integrando visualizaciones de tendencias en tiempo real e historiales climáticos.

◆ Lo que se sostiene igual:

- **El problema de origen:** La necesidad de eliminar el registro manual desde el Monitor Serie y evitar la pérdida de las series temporales de la estación ambiental.
- **La arquitectura base de ingesta y persistencia:** El uso del microcontrolador Arduino UNO R3 enviando tramas por puerto serie USB hacia un script local en Python 3.12, que persiste los registros de forma estructurada en la base de datos relacional MySQL (*estacionmeteorologica_proa*), utilizando un buffer local en formato CSV como mecanismo de resiliencia ante caídas de conexión.
- **Las decisiones de diseño clave:** La separación estricta de responsabilidades entre el hardware (dedicado a lectura y actuación física) y el software (Python/MySQL para lógica y alertas), así como la

decisión de evitar *triggers* a nivel de base de datos para no degradar el rendimiento del motor relacional ni acoplar llamadas HTTP externas.

○

◆ Lo que cambió o evolucionó:

- **Integración y tipificación del sensor MQ-2 (Gas/Humo):** Mientras que en la fase inicial el enfoque fue primordialmente térmico y de humedad, en la Evidencia 2 se consolidó la captura del sensor analógico MQ-2, modificando la estructura de la tabla mediciones (ALTER TABLE mediciones ADD COLUMN gas INT AFTER humedad) e incorporando la clasificación categórica del nivel de riesgo en PPM por software.
- **Incorporación del usuario agrícola (HU-AGRO-01):** Se identificó y formalizó la necesidad del sector rural circundante de acceder a una plataforma en línea con gráficos interactivos de tendencias históricas (temperatura, humedad, gases) para fundamentar decisiones de siembra y riego.
- **Mecanismos defensivos en el código:** Se profundizó la canalización defensiva en Python agregando filtros automáticos de descarte para tramas corruptas ("ERROR" o longitud distinta a 3 elementos) y validación visual con la librería colorama en consola antes de la integración con Telegram.
- **Migración de la interfaz de interacción:** Se redefinió el canal de consulta para la comunidad educativa y los productores agrícolas, reemplazando la propuesta inicial de WhatsApp por Telegram. Esta decisión se fundamenta en que Telegram es un ecosistema libre y completamente gratuito, lo que elimina la dependencia de intermediarios de pago (como las API comerciales de WhatsApp Business / Twilio) y garantiza la sustentabilidad económica del proyecto a largo plazo para la institución. Además, la Telegram Bot API ofrece capacidades superiores para el desarrollo de chatbots: permite gestionar comandos nativos estructurados (como /clima, /estado o /reporte), integrar teclados interactivos (Inline Keyboards), procesar eventos mediante Webhooks de manera directa y enviar alertas o gráficos generados por la estación con menor latencia y sin restricciones operativas.

◆ Aprendizajes al trabajar el dato:

- **Ruido eléctrico y fallas de hardware:** Se constató que la señal serie física sufre interferencias o desconexiones (ej. cables Dupont sueltos), por lo que es indispensable filtrar y limpiar los bytes crudos en la memoria volátil de Python antes de ejecutar sentencias INSERT para evitar contaminar la base de datos de la escuela.
- **Manejo de sensores analógicos de espectro amplio:** Aprendimos que el sensor MQ-2 no identifica la firma química exacta del gas; por ende, la severidad del riesgo debe modelarse en software mediante rangos de PPM (<250 Aire Normal, 250-450 Alerta Media, >450 Alerta Crítica) sin recargar ni re-flashear el microcontrolador Arduino.
- **Optimización de espacio:** Confirmamos que almacenar las variables numéricas como enteros (INT) en MySQL optimiza el almacenamiento en disco para soportar registros continuos cada 2 segundos sin sobrecargar la infraestructura del colegio.

◆ **Aprendizajes al construir la historia de usuario (HU-AGRO-01):**

- **Resiliencia en entornos rurales:** Al evaluar la historia bajo el criterio INVEST, comprendimos que la tolerancia a fallos de conectividad (buffer local y sincronización automática) es una condición funcional indispensable para que la solución sea verdaderamente valiosa en el campo.
- **Granularidad e Independencia:** Validamos que la captura continua en segundo plano y la consulta visual web pueden empaquetarse de manera autónoma, permitiendo negociar el tipo de gráficos e indicadores sin bloquear el desarrollo de otros módulos.

◆ **Ajustan de cara a lo que sigue**

- **Despliegue e integración final del Chatbot (Telegram Bot API + FastAPI):** Transicionar de la lectura y validación visual local en consola (colorama) a la puesta en producción de un servidor Webhook desacoplado mediante FastAPI e integrado con la Telegram Bot API. Esto permitirá procesar en tiempo real los comandos nativos de Telegram (/clima, /estado, /reporte) e interactuar de forma fluida con la comunidad educativa y los productores rurales.
- **Orquestación y disponibilidad continua (24/7):** Configurar el servicio demonio de alertas proactivas (para temperaturas $>32^{\circ}\text{C}$ / $<8^{\circ}\text{C}$ y niveles críticos de gas) como un proceso persistente systemd e implementar la contenedorización con Docker para asegurar un uptime objetivo del 99.5%.
- **Agregaciones estadísticas avanzadas:** Utilizar Pandas en el backend para automatizar las agregaciones diarias de máximas, mínimas y cálculo de la pendiente de temperatura de la última hora sobre MySQL, alimentando tanto las respuestas del bot de Telegram como la interfaz web de tendencias.